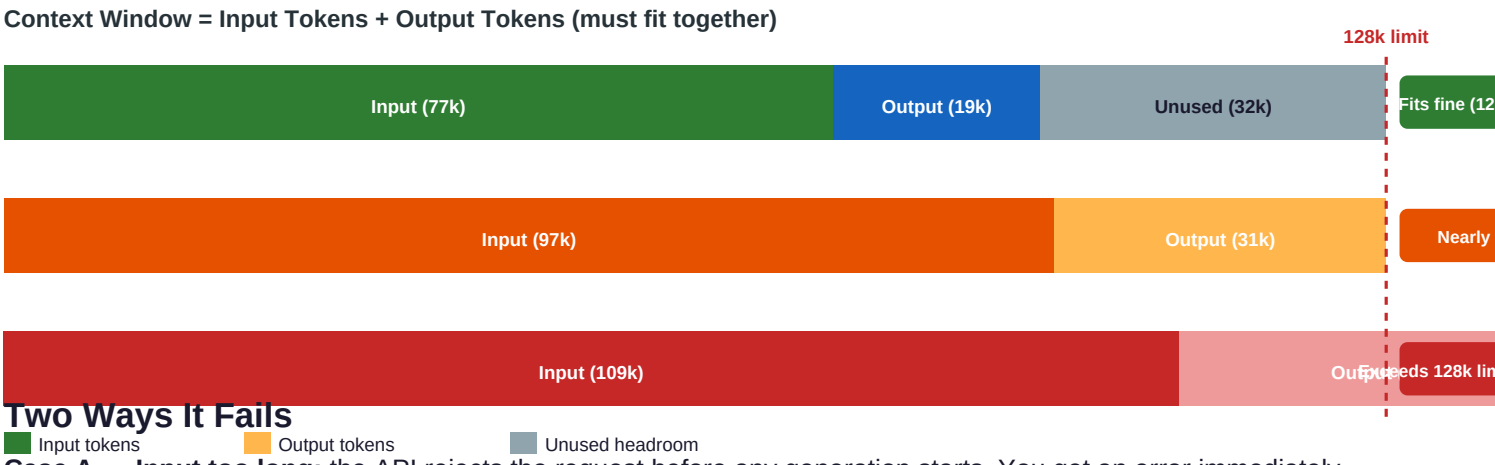
Context Windows: What Happens When You Exceed Them

And how to handle it in Python — with correct examples

> **Context window** = the total number of tokens a model can process in one call. It is a shared budget: **input tokens + output tokens must both fit inside it**. GPT-4o has a 128k-token window. Claude 3.5 Sonnet has a 200k-token window.

The Context Window Budget



Two Ways It Fails

Case A — Input too long: the API rejects the request before any generation starts. You get an error immediately with no output.

```
import openai

client = openai.OpenAI()

# GPT-4o limit: 128,000 tokens

# Sending 135,000 tokens of input -> rejected immediately

response = client.chat.completions.create(

model="gpt-4o",

messages=[{"role": "user", "content": very_long_text}]

)

# openai.BadRequestError: 400 Bad Request

# This model's maximum context length is 128,000 tokens.

# Your input contained 135,412 tokens. (135,412 > 128,000)
```

Case B — Output gets cut off: the input fits, but the model runs out of room while generating its response. It stops mid-sentence. No error is raised — you get a partial answer with **finish_reason == 'length'** instead of **'stop'**.

```
response = client.chat.completions.create(
```

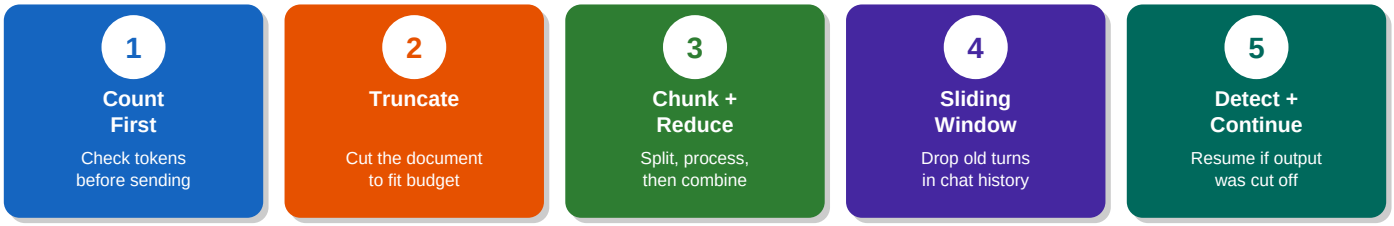
```
model="gpt-4o",
messages=[{"role": "user", "content": prompt}],
max_tokens=4096
)

choice = response.choices[0]
print(choice.finish_reason) # "length" <- output was cut off
# "stop" <- model finished naturally
print(choice.message.content) # ...ends mid-sentence
```

Common Model Context Limits (as of mid-2025)

Model	Context Window	Max Output	Input Budget (leaving 4k for output)
GPT-4o	128,000 tokens	16,384 tokens	~111,616 tokens safe input
GPT-4o mini	128,000 tokens	16,384 tokens	~111,616 tokens safe input
Claude 3.5 Sonnet	200,000 tokens	8,096 tokens	~191,904 tokens safe input
Claude 3 Haiku	200,000 tokens	4,096 tokens	~195,904 tokens safe input
Gemini 1.5 Pro	1,000,000 tokens	8,192 tokens	~991,808 tokens safe input

5 Strategies — Pick the Right One for Your Situation



Handling Strategies

Concrete Python examples for each approach

1 Count Tokens Before Sending

When to use: Always — a good habit before every API call

Count the tokens in your prompt before sending it to the API. If you exceed the budget, raise a clear error with the overage size so you know exactly how much to trim.

```
import tiktoken
import openai

MODEL = 'gpt-4o'
CONTEXT_LIMIT = 128_000 # GPT-4o hard limit
OUTPUT_RESERVE = 4_000 # tokens to keep free for the model's reply
INPUT_BUDGET = CONTEXT_LIMIT - OUTPUT_RESERVE # = 124,000

def count_tokens(text: str) -> int:
enc = tiktoken.encoding_for_model(MODEL)
return len(enc.encode(text))

def safe_chat(prompt: str) -> str:
token_count = count_tokens(prompt)

if token_count > INPUT_BUDGET:
overage = token_count - INPUT_BUDGET
raise ValueError(
f"Prompt is {token_count:,} tokens -- exceeds budget of "
f"{INPUT_BUDGET:,}. Trim by ~{overage:,} tokens."
)
# Example message if prompt = 130,000 tokens:
# ValueError: Prompt is 130,000 tokens -- exceeds budget
# of 124,000. Trim by ~6,000 tokens.

client = openai.OpenAI()
response = client.chat.completions.create(
model=MODEL,
messages=[{"role": "user", "content": prompt}],
max_tokens=OUTPUT_RESERVE
)
return response.choices[0].message.content
```

Key point: 130,000 > 124,000 (budget), not > 128,000 (hard limit). Always leave a buffer for output. Sending 127,000 tokens of input would only leave 1,000 tokens for the model's reply — often not enough.

2 Truncate the Document

When to use: Summaries, log files, articles — where start or end is expendable

When the document is too long, encode it to tokens, slice to fit the budget, then decode back to text. Simple and fast — but you permanently lose content outside the slice.

```
import tiktoken
import openai

MODEL = 'gpt-4o'
CONTEXT_LIMIT = 128_000
OUTPUT_RESERVE = 4_000

def truncate_to_fit(system: str, document: str, question: str) -> str:
enc = tiktoken.encoding_for_model(MODEL)

# Count tokens used by the fixed parts of the prompt
overhead = 60 # role markers, formatting
fixed_tokens = len(enc.encode(system + question)) + overhead
doc_budget = CONTEXT_LIMIT - OUTPUT_RESERVE - fixed_tokens
# e.g. 128,000 - 4,000 - 1,200 - 60 = 122,740 tokens for the doc

doc_tokens = enc.encode(document)
if len(doc_tokens) > doc_budget:
doc_tokens = doc_tokens[:doc_budget] # keep the start
document = enc.decode(doc_tokens)
document += "\n\n[Document truncated to fit context window.]"

client = openai.OpenAI()
response = client.chat.completions.create(
model=MODEL,
messages=[
{"role": "system", "content": system},
{"role": "user", "content": document + "\n\n" + question}
],
max_tokens=OUTPUT_RESERVE
)
return response.choices[0].message.content
```

Strategies 3 & 4

Chunk + Reduce for long documents — Sliding Window for chat history

3 Chunk + Reduce (Map-Reduce)

When to use: Long books, transcripts, reports — where the middle matters too

Split the document into chunks that each fit within the context window, summarise each chunk independently (map), then combine all summaries into one final answer (reduce). The model never needs to see the whole document at once.

```
import tiktoken
import openai

MODEL = 'gpt-4o'
CONTEXT_LIMIT = 128_000
CHUNK_TOKENS = 20_000 # tokens per chunk (well under the 128k limit)
OUTPUT_PER_CHUNK = 1_000 # reserved per chunk summary

def split_into_chunks(text: str) -> list[str]:
enc = tiktoken.encoding_for_model(MODEL)
tokens = enc.encode(text)
chunks = []
for i in range(0, len(tokens), CHUNK_TOKENS):
chunk_tokens = tokens[i : i + CHUNK_TOKENS]
chunks.append(enc.decode(chunk_tokens))
return chunks

def summarise_long_document(document: str) -> str:
client = openai.OpenAI()
chunks = split_into_chunks(document)
print(f'Document split into {len(chunks)} chunks')
# e.g. 80,000-token doc -> 4 chunks of 20,000 tokens each

# MAP: summarise each chunk independently
summaries = []
for i, chunk in enumerate(chunks):
resp = client.chat.completions.create(
model=MODEL,
messages=[{
"role": "user",
"content": f"Summarise this section:\n\n{chunk}"
}],
```

```
max_tokens=OUTPUT_PER_CHUNK # 1,000 tokens per summary
)
summaries.append(resp.choices[0].message.content)
print(f" Chunk {i+1}/{len(chunks)} done")

# REDUCE: combine all summaries -> one final summary
combined = "\n\n---\n\n".join(summaries)
final = client.chat.completions.create(
model=MODEL,
messages=[{
"role": "user",
"content": f"These are summaries of each section:\n\n{combined}"
"\n\nWrite one cohesive final summary."
}],
max_tokens=2_000
)
return final.choices[0].message.content
```

Limitation: The model cannot reason across chunks. If a key fact in chunk 1 is needed to understand chunk 3, it will be missed. Use RAG (Retrieval-Augmented Generation) when cross-document reasoning matters.

4 Sliding Window (Chat History Trimming)

When to use: Any multi-turn chatbot — history grows indefinitely

In a chat loop, every message gets appended to history. After many turns the accumulated history can exceed the context window. The fix: drop the oldest non-system messages until the conversation fits again.

```
import tiktoken
import openai

MODEL = 'gpt-4o'
CONTEXT_LIMIT = 128_000
OUTPUT_RESERVE = 4_000
INPUT_BUDGET = CONTEXT_LIMIT - OUTPUT_RESERVE # = 124,000

def token_count_of_messages(messages: list[dict]) -> int:
enc = tiktoken.encoding_for_model(MODEL)
total = 0
for msg in messages:
total += len(enc.encode(msg['content'])) + 4 # 4 = role overhead
return total

def trim_history(messages: list[dict]) -> list[dict]:
```

```
system_msgs = [m for m in messages if m['role'] == 'system']
other_msgs = [m for m in messages if m['role'] != 'system']

# Drop oldest non-system messages until we fit inside INPUT_BUDGET
# Always keep at least the last user message (other_msgs[-1])
while (token_count_of_messages(system_msgs + other_msgs) > INPUT_BUDGET
and len(other_msgs) > 1):
other_msgs.pop(0) # remove the oldest turn

return system_msgs + other_msgs

# ---- Chat loop ----
client = openai.OpenAI()
history = [{"role": "system", "content": "You are a helpful assistant."}]

while True:
user_input = input('You: ')
history.append({'role': 'user', 'content': user_input})

trimmed = trim_history(history) # trim BEFORE sending
dropped = len(history) - len(trimmed)
if dropped > 0:
print(f'[{dropped} old messages dropped to stay under 124k budget]')

response = client.chat.completions.create(
model=MODEL, messages=trimmed, max_tokens=OUTPUT_RESERVE
)
reply = response.choices[0].message.content
history.append({'role': 'assistant', 'content': reply})
print(f'Assistant: {reply}')
```

Strategy 5 + Quick-Reference Guide

Detect truncated output and resume — plus when to use each strategy

5 Detect Truncated Output and Continue

When to use: Long code generation, detailed reports — where output length is unpredictable

When the model's output gets cut off (finish_reason == 'length'), append what it produced so far as an assistant turn, then ask it to continue. Repeat until it finishes naturally (finish_reason == 'stop').

```
import openai

MODEL = 'gpt-4o'

MAX_OUTPUT = 4_096 # tokens per generation call

MAX_CONTINUATIONS = 4 # stop after 4 continuations to avoid loops

def generate_full_response(prompt: str) -> str:

client = openai.OpenAI()

messages = [{'role': 'user', 'content': prompt}]

full_response = ''

for attempt in range(MAX_CONTINUATIONS + 1):

response = client.chat.completions.create(

model=MODEL,

messages=messages,

max_tokens=MAX_OUTPUT # 4,096 per call

)

choice = response.choices[0]

full_response += choice.message.content

if choice.finish_reason == 'stop':

break # model finished naturally -- we are done

elif choice.finish_reason == 'length':

# Output was cut off. Append what we got and ask to continue.

print(f'Output cut off at attempt {attempt + 1}, continuing...')

messages.append({

'role': 'assistant',

'content': choice.message.content

})

messages.append({

'role': 'user',

'content': 'Continue exactly where you left off.'
```

```
})
else:
break # content_filter or other reason -- do not retry

return full_response
```

Watch out: each continuation appends to the message history, so the context window fills up across continuations too. After 4-5 continuations, count total tokens and consider switching to a model with a larger output limit.

Which Strategy Should I Use?

Situation	Best strategy	Limitation
You want to prevent errors proactively	**1 — Count first**	Tells you the problem; you still have to fix it
Summarising news/logs where beginning matters most	**2 — Truncate**	Permanently loses content outside the slice
Summarising a full book or long report	**3 — Chunk + Reduce**	Loses cross-chunk reasoning; slower (multiple calls)
Multi-turn chatbot that runs indefinitely	**4 — Sliding Window**	Model forgets old turns; consider periodic summarisation
Generating long code, essays, or reports	**5 — Detect + Continue**	History grows per continuation; cap at 4-5 rounds
Need reasoning across the entire document	**Use a larger-context model (Gemini 1M, Claude 200k)**	Cost increases with context size

Key Takeaways Checklist

■	Know the context limit of every model you use (GPT-4o = 128k, Claude 3.5 = 200k)
■	Remember: input + output must both fit -- always reserve budget for output tokens
■	An input of 195,000 tokens is fine on Claude (200k limit). It errors only above 200k.
■	A truncated output has finish_reason='length', NOT an exception -- you must check for it
■	Install tiktoken (OpenAI) and count tokens before every large prompt
■	For chat bots: trim history before sending, not after getting an error
■	Chunk + Reduce loses cross-chunk reasoning -- use RAG when that matters
■	Cap continuation loops at 3-5 rounds -- context fills up across continuations too